

Feuille de travaux dirigés n° 4

Le modèle mémoire de Java **— CORRECTION —**

Exercice 1 (Volatilité).

Les trois codes suivants s'exécutent en parallèle, avec initialement, $x = y = z = 0$.

```
/****** Thread 1 *****/  
x = 1;  
y = 1;  
  
/****** Thread 2 *****/  
if (y == 1)  
z = 1;  
  
/****** Thread 3 *****/  
if (z == 1)  
System.out.println(x);
```

- 1a. Quelles sont les sorties possibles si x , y et z sont déclarés **volatile**?
- 1b. Quelles variables faut-il déclarer **volatile** pour éliminer tout data race?

Exercice 2 (Singleton).

On rappelle le code séquentiel du patron de conception *Singleton*.

```
class Singleton {  
    private static Singleton instance = null;  
    private Singleton() {}  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

- 2a. Donnez une exécution avec deux threads où deux instances sont créées.
- 2b. Corrigez le programme. Il faudra réduire l'usage des verrous au maximum.

Exercice 3 (Identification de *data races*).

On examine le programme Java suivant. Les trois variables partagées sont initialisées à `x = 0` et `flag = true`. On suppose qu'aucun autre thread n'intervient et que les réveils intempestifs possibles de `wait()` sont correctement gérés par la boucle `while`. On raisonne sous l'hypothèse de *cohérence séquentielle* pour les questions 1–3.

```
1  final class Program {
2      static final Object o = new Object();
3      static int x = 0;
4      static boolean flag = true;
5
6      public static void main(String[] args) throws Exception {
7          Thread t = new Thread(() -> {
8              x = 1;
9              synchronized (o) {
10                 flag = false;
11                 o.notify();
12             }
13             x = 2;
14         });
15
16         synchronized (o) {
17             t.start();
18             while (flag) {
19                 o.wait();
20             }
21             int a = x;
22         }
23         int b = x;
24     }
25 }
```

- 3a. Donnez la machine à états du programme.
- 3b. Déduisez-en toutes les exécutions séquentiellement cohérentes. Quelles sont les valeurs possibles de *a* et *b*?
- 3c. Identifiez toutes les *data races* présentes dans le programme.
- 3d. Donnez deux corrections *minimales* et *distinctes* qui éliminent toutes les *data races* :
 - en ne modifiant que des qualificatifs (ex. `volatile`) dans les déclarations;
 - en ne modifiant que la synchronisation (ex. blocs `synchronized`).
- 3e. Pour chacune des deux corrections, quelles sont alors les valeurs possibles de *a* et *b*?